RAG Benchmark — note di design su retrieval ed esperimenti

Oct 23, 2026 · @Diego Zendata

Scopo

Raccoglie le decisioni e i chiarimenti emersi in una sessione di lavoro su retrieval, esperimenti e tracciamento degli agenti nella piattaforma di benchmarking RAG. Lo stato del codice descritto è quello al 23 ottobre 2026: tutto ciò che riguarda esperimenti, esecuzioni e metriche esiste solo nei documenti di design, non nel codice.

Ogni sezione è autonoma: la prima riga dice la conclusione, il resto la motivazione. L'ultima sezione elenca le decisioni prese e i punti ancora da chiudere.

Stato del progetto

La piattaforma sa costruire un indice vettoriale riproducibile e interrogarlo, ma non sa ancora misurarlo: mancano QA set, esperimenti e metriche.

Milestone	Stato	Cosa c'è / cosa manca
1 — Acquisizione e fondamenta	completa (solo upload)	upload con deduplica per hash, corpus, snapshot, profili, materializzazione per documento su Temporal, ispezione chunk e parse, UI
2 — Sorgenti remote	non iniziata	modelli presenti; nessun connettore filesystem, S3 o SharePoint, nessuna ri-sincronizzazione
3 — Indicizzazione e retrieval	quasi completa	index build su Chroma e Qdrant, profili di retrieval, `POST /retrieve` con ricerca densa e UI "Prova query"; mancano esperimenti di puro retrieval e metriche documentali (Sprint 07)
4 — Authoring	non iniziata	risposte attese, citazioni gold, legami di evidenza, metriche a livello chunk

Milestone	Stato	Cosa c'è / cosa manca
5 — Q&A end-to-end	non iniziata	contratto `/answer`, sistemi di risposta, Langfuse, judge, verifica umana
6 — Confronto e trasferimento	non iniziata	leaderboard, confronto affiancato, wizard di trasferimento, significatività

Componenti disponibili: parser `pypdf` e `pdfplumber`, chunker `recursive` e `markdown`, embedder OpenAI e Bedrock, vector store Chroma e Qdrant. Formato supportato: solo PDF.

Sprint 06-07 e dove gira il retrieval

La Milestone 3 si chiude in due sprint, e il retrieval gira nell'API, non nel worker. I file di piano sono in `sprint/` (cartella esclusa da git).

Sprint	Tema	Esito verificabile	Stato
06	profili di retrieval, ricerca densa su Chroma e Qdrant, `POST /retrieve`, UI di prova query	query su un indice `ready` da browser e da `curl`	consegnato
07	QA set minimo (domande + documenti gold), esperimenti di puro retrieval, run su Temporal, metriche documentali	dal CSV dei gold alla tabella delle metriche per variante	da fare

Perché nell'API. Il tool di retrieval è sincrono e verrà chiamato dagli agenti molte volte per domanda. Passare da un'activity Temporal aggiungerebbe latenza senza vantaggi: in una query non c'è nulla da ritentare o riprendere.

Conseguenza. Il lato lettura di embedder e vector store è codice del backend (`backend/src/retrieval/`); il lato scrittura resta nel worker. La direzione delle dipendenze resta `temporal → backend`. L'API ora ha bisogno delle credenziali degli embedder e di raggiungere Chroma e Qdrant.

Lo Sprint 07 anticipa un pezzo della Milestone 4: domande e documenti gold, con le tabelle definitive del data model, per avere i primi numeri a livello documento senza annotare chunk.

Rimozione dei filtri sui metadati dal profilo

`metadata_filters` è stato eliminato da `retrieval_profiles` (migrazione `c3e9a5d1f47b`): un filtro sui documenti nel profilo rompe la confrontabilità.

Il principio. Lo spazio di ricerca fa parte della misura: recall@5 su 50 documenti e su 200 non è la stessa metrica. Lo spazio di ricerca lo fissa lo snapshot di corpus, uguale per tutte le varianti di un esperimento.

Cosa faceva di male il filtro nel profilo:

- era uno snapshot mascherato: due varianti con profili diversi avrebbero cercato in documenti diversi dentro lo stesso esperimento;
- poteva escludere documenti gold, dando recall 0 per costruzione invece che per colpa del retriever;
- non serviva: restringere i documenti è già gratuito creando uno snapshot.

Cosa è cambiato. Tolto da modello, API, servizio di retrieval, UI e documentazione. La migrazione ricalcola gli hash dei profili e tiene il più vecchio fra quelli che differivano solo per i filtri. I lettori di Chroma e Qdrant conservano una restrizione per `document_id`, oggi senza chiamanti.

Dove avrebbero senso i filtri, se servissero:

Dove	Filtro	Perché
profilo, sui chunk	`modality`, lunghezza minima, esclusione per heading	è una scelta di strategia da misurare e non cambia l'identità dei chunk, quindi il gold resta valido
singola chiamata a `/retrieve`	metadati del documento	li sceglie l'agente in base alla domanda: è comportamento dell'agente da misurare, va registrato con la chiamata
singola chiamata, prova query	`document_ids`	debug dalla UI

Pipeline di retrieval custom: due operazioni, search e fetch

La piattaforma offre due operazioni semplici, `search` e `fetch`, entrambe tracciate; la logica che le combina ("cerca i top-k, poi prendi i vicini") sta in chi le chiama. È una proposta, non implementata, e sostituisce l'idea precedente di una catena di passi dentro il profilo di retrieval.

	`search` (l'attuale `/retrieve`)	`fetch` (nuova)
Input	testo della query + profilo di retrieval	filtri espliciti, nessuna query
Cosa fa	embedding, ricerca per similarità, soglia, in futuro rerank	restituisce chunk precisi
Ordine e score	rank e score di similarità	ordine deterministico (documento, ordinale), nessuno score
Dove legge	vector store + Postgres	solo Postgres

Perché `fetch` legge solo da Postgres. Testo, `document_id`, `ordinal` e pagina dei chunk sono già nel database. Il comportamento è identico per ogni vector store e il loro lato lettura non va toccato. I vicini di un chunk sono una query su `ordinal ± finestra` nello stesso `chunk_result`.

Filtri di `fetch`, schema chiuso: `chunk_ids`; `document_id` + intervallo di `ordinal`; `document_id` + intervallo di pagine; un tetto massimo di chunk restituiti.

Vincolo lato server. `fetch` restituisce solo chunk dell'indice dell'esecuzione: la configurazione di chunking del build e i documenti del suo snapshot. Il server interseca i filtri con quell'insieme (è la query di `IndexBuildRepository.list_chunks`). Senza questo, un agente potrebbe uscire dallo spazio di ricerca dell'esperimento.

Tracciamento. Entrambe le operazioni scrivono nelle stesse tabelle, legate all'esecuzione dal token:

- `retrieval_calls`: nuova colonna `kind` (`search` | `fetch`), `query_text` nullable, nuova colonna `request` JSONB con filtri o profilo; `call_index` unico per esecuzione e comune ai due tipi, così si vede la sequenza reale;
- `retrieval_hits`: nuovo stage `fetched`, `score` nullable, `rank` = posizione nella risposta;

- facoltativo `parent_call_id` sulla fetch, se il chiamante dichiara da quale ricerca nasce: utile al debug, non alle metriche.

Effetto sulle metriche:

- i chunk recuperati con fetch entrano nell'insieme recuperato; la tabella per esecuzione × chunk ne registra la provenienza (search, fetch o entrambe);
- recall@k, MRR e primo rilevante si calcolano solo sulle chiamate di search: una fetch non ha un ordine di rilevanza, e un chunk preso per id non è merito del retriever;
- il gonfiamento dell'unione peggiora: va riportata sempre la dimensione dell'insieme, o si confronta a parità di budget.

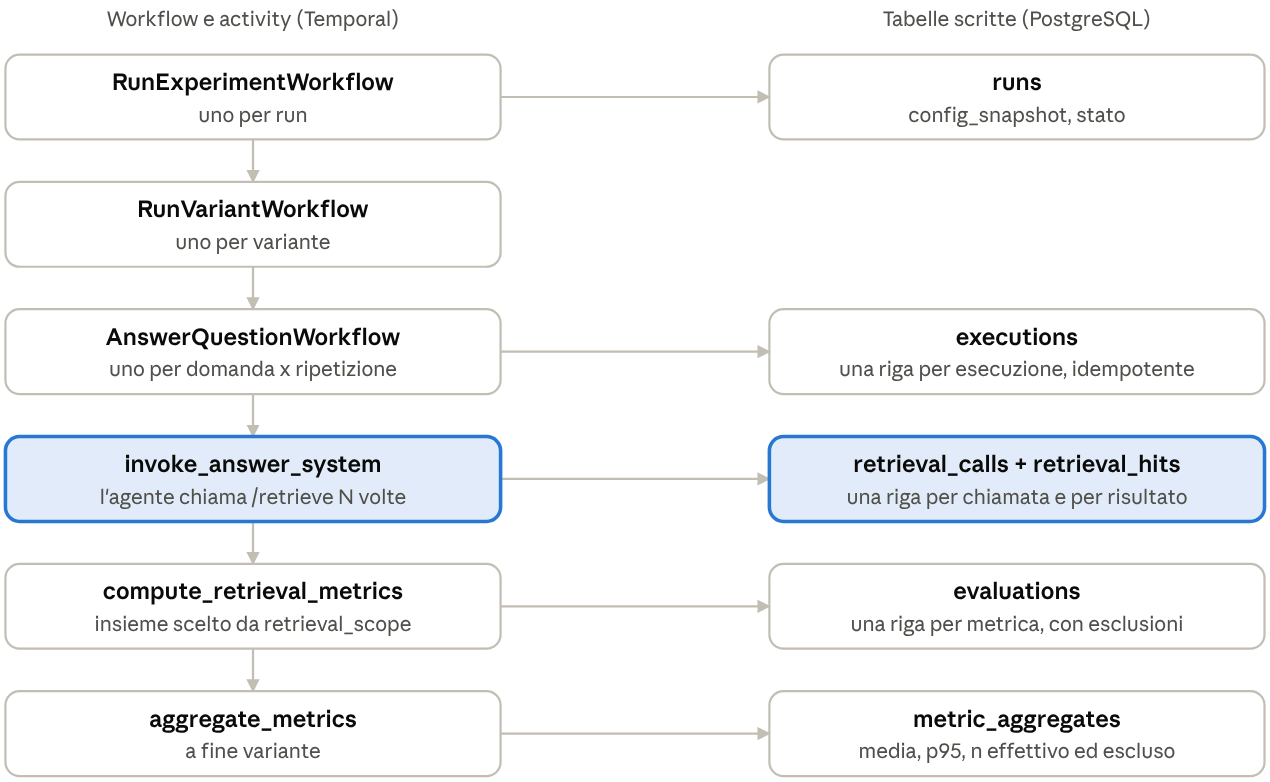
Il costo. La pipeline esce dal profilo e finisce nel chiamante. Negli esperimenti Q&A è il sistema di risposta, e si confronta variando il profilo di risposta. Negli esperimenti di puro retrieval oggi non c'è un chiamante: serve un componente `retrieval_system` che usa le due operazioni e restituisce chunk invece di una risposta. Quello builtin predefinito fa una sola search con il profilo, come previsto nello Sprint 07; la variante diventa indice × profilo di retrieval × sistema di retrieval.

Cosa cambierebbe nei documenti di design: requisiti sull'osservazione del retrieval (due tipi di chiamata), data model (`retrieval_calls` e `retrieval_hits` come sopra), contratto `/answer` (una base URL con `/search` e `/fetch` al posto di un solo `retrieval_endpoint`), Sprint 07 (se `fetch` entra subito o dopo).

Tracciamento del retrieval di un agente

Ogni ricerca dell'agente passa dal tool della piattaforma, che la registra lato server legandola all'esecuzione; le metriche si calcolano poi su uno dei tre insiemi (recuperato, in contesto, citato). Nulla di questo è implementato: le tabelle esistono solo nel data model e `/retrieve` oggi usa `execution_id` solo nel log.

Ogni ricerca dell'agente diventa una riga legata all'esecuzione



flusso di un run Q&A · workflow e tabelle scritte

A sinistra la gerarchia dei workflow, a destra le tabelle che ogni livello scrive; in evidenza il punto in cui nasce la traccia del retrieval.

Esempio usato in tutta la sezione. Domanda Q1 "Qual è la penale per ritardo nella consegna?", documento gold `Capitolato_A.pdf`. Variante V1 = indice IB1 × profilo `top_k=5` × agente interno. Una ripetizione. L'agente fa due ricerche.

1. **Preparazione.** `experiments` fissa tipo, snapshot di corpus, snapshot del QA set, ripetizioni e `retrieval_scope`. `variants` contiene V1 (indice, profilo di retrieval, profilo di risposta).
2. **Lancio del run.** `runs` salva `config_snapshot` (copia integrale della configurazione) e avvia i workflow: `RunExperimentWorkflow` → un `RunVariantWorkflow` per variante → un `AnswerQuestionWorkflow` per domanda × ripetizione.
3. **Esecuzione.** Riga E1 in `executions` per (run, V1, Q1, ripetizione 0). Il vincolo unico su questi quattro campi rende il rilancio idempotente. Tutto ciò che segue è legato a E1.

4. **L'agente lavora.** L'activity `invoke_answer_system` chiama `POST /answer` passando `retrieval_endpoint` e un `retrieval_token` valido solo per E1. Ogni chiamata a `/retrieve` scrive una riga in `retrieval_calls` e una per risultato in `retrieval_hits`. Alla risposta, E1 riceve testo, `usage` e `timings`; le `citations` mettono `cited = true` sui risultati corrispondenti.
5. **Metriche di retrieval.** `compute_retrieval_metrics` marca ogni risultato con `relevant_document` e, se c'è il legame di evidenza, `relevant_chunk`; sceglie l'insieme secondo `retrieval_scope`; scrive una riga per metrica in `evaluations`, con `excluded` e motivo quando la domanda non è valutabile.
6. **Risposta e Langfuse.** Traccia con uno span per chiamata di retrieval e per la generazione; il judge valuta la risposta; i punteggi tornano in `evaluations`.
7. **Aggregazione.** A fine variante `metric_aggregates` riceve media, deviazione, mediana, p95, `n_effective`, `n_excluded`. Da qui leggono leaderboard e UI.

Le righe scritte al passo 4:

`retrieval_calls`	`call_index`	`query_text`
C1	0	penale ritardo
C2	1	penali consegna articolo 12

`retrieval_hits`	chiamata	chunk	`rank`	`score`
	C1	ch_B7 (Capitolato_B)	1	0,81
	C1	ch_A3 (Capitolato_A)	2	0,78
	C2	ch_A12 (Capitolato_A)	1	0,90
	C2	ch_A3 (Capitolato_A)	2	0,85

L'agente cita ch_A12. Insiemi di E1: recuperato = {ch_B7, ch_A3, ch_A12, …}; citato = {ch_A12}; in contesto = nessuno, perché nessuno lo dichiara (vedi la sezione su `in_context`).

Problemi aperti con più ricerche

Cinque punti del design non reggono quando un'esecuzione fa più ricerche; vanno chiusi prima dello Sprint 07 perché cambiano lo schema.

#	Problema	Proposta
1	Recall@k, MRR e "primo rilevante" sull'unione non sono definiti: N chiamate danno N liste con rank 1..k, e gli score di query diverse non sono confrontabili	sull'unione solo metriche di insieme (recall, precision, senza @k) con la dimensione dell'insieme; @k e MRR per chiamata, aggregati come migliore e prima chiamata; nuova metrica: `call_index` del primo risultato rilevante
2	L'unione cresce con il numero di chiamate: 10 ricerche da 5 coprono fino a 50 chunk e battono un top-5 senza essere un retriever migliore	riportare sempre dimensione dell'unione e numero di chiamate, o confrontare a parità di budget di chunk
3	`in_context` non è ricavabile da un agente esterno: il contratto `/answer` non ha un campo per i chunk messi nel prompt	campo opzionale `context: [chunk_id]`; se assente, ambito `in_context` escluso con motivo invece di dare 0
4	`in_context` e `cited` sono proprietà del chunk nell'esecuzione, non del singolo risultato: lo stesso chunk in tre chiamate dà tre righe, e una citazione di un chunk mai recuperato non ha riga su cui stare	`retrieval_hits` resta il log grezzo; nuova tabella per esecuzione × chunk (es. `execution_chunks`) con `retrieved`, `first_call_index`, `best_rank`, `in_context`, `cited` e flag di rilevanza
5	Senza token per esecuzione, `/retrieve` prende per buono l'`execution_id` nel corpo: chiamate senza id o con id sbagliato spariscono dalle metriche	anticipare il token allo Sprint 07, insieme a `executions`

Ripetizioni

Le ripetizioni (`experiments.repetitions`, default 1) dicono quante volte, dentro lo stesso run, ogni domanda viene eseguita su ogni variante; servono a misurare sistemi non deterministici.

Ogni ripetizione è una riga distinta di `executions` (`repetition` = 0, 1, 2, …). Con 150 domande, 2 varianti e 3 ripetizioni un run produce 900 esecuzioni, tutte con la stessa configurazione congelata.

A cosa servono:

- una media per domanda più stabile;
- la varianza fra ripetizioni, metrica di robustezza a sé ("giusto 3 volte su 3" contro "2 su 3");
- la base per dire se la differenza fra due varianti è significativa o rumore.

Quando non servono. Nel puro retrieval la ricerca densa è deterministica: si lascia 1.

Ripetizioni o più run? Dentro un run le ripetizioni condividono la configurazione congelata e sono eseguite insieme; due run separati possono differire se nel frattempo è cambiato qualcosa fuori dalla piattaforma, per esempio un agente esterno ridistribuito.

Da fissare nell'aggregazione. Media delle ripetizioni per domanda prima, poi media fra domande. Contare 900 esecuzioni come 900 campioni indipendenti invece di 150 domande restringe troppo gli intervalli di confidenza.

L'insieme "in contesto"

`in_context` indica i chunk che l'agente, fra quelli ricevuti dal tool, ha davvero messo nel prompt dell'LLM; è nel design ma oggi nessuno può impostarlo.

Dove compare: nei requisiti, fra i tre insiemi dell'osservazione del retrieval; nel data model, come colonna `retrieval_hits.in_context` accanto a `cited`, e come valore di `experiments.retrieval_scope` (`retrieved`, `in_context`, `cited`).

I tre insiemi, con un agente che recupera 10 chunk in due ricerche:

Insieme	Chunk	Domanda a cui risponde
recuperato	tutti e 10	il retrieval ha trovato l'evidenza?
in contesto	i 3 passati all'LLM dopo filtro o riordino	l'evidenza trovata è arrivata al modello?
citato	quello dichiarato nella risposta	l'attribuzione è corretta?

Evidenza recuperata ma non in contesto: il retrieval ha funzionato, l'agente l'ha scartata. In contesto ma risposta sbagliata: il problema è la generazione.

Perché è un buco. Il recuperato la piattaforma lo sa da sé; il citato arriva dalle `citations` di `/answer`. Il contesto è una decisione interna all'agente e il contratto non ha un campo per dichiararla: per un agente esterno il flag resta sempre `false`, e un esperimento con ambito `in_context` darebbe recall 0 senza che nessuno se ne accorga. Per un sistema di risposta builtin il problema non c'è, perché gira nel worker. La proposta è al punto 3 dei problemi aperti.

Esperimento e varianti

L'esperimento fissa ciò che deve restare uguale perché un confronto abbia senso; le varianti sono le configurazioni messe a confronto.

Esperimento (uguale per tutte le varianti)	Variante (ciò che si confronta)
snapshot di corpus, cioè lo spazio di ricerca	indice: configurazione di chunking × embedder × vector store
snapshot del QA set, cioè le domande	profilo di retrieval
tipo (retrieval o Q&A)	profilo di risposta: agente, modello, prompt, temperatura
ripetizioni, ambito delle metriche	se è la baseline

Esempio. "Docling o pypdf, e con quale embedder?" = un esperimento su "Capitolati 2026" e QA set v3, con quattro varianti: pypdf + OpenAI (baseline), pypdf + Bedrock, Docling + OpenAI, Docling + Bedrock. Un solo lancio produce quattro righe confrontabili.

Cosa si perde mettendo tutto sull'esperimento (un esperimento per configurazione):

1. La garanzia strutturale di confrontare la stessa cosa: nulla impedirebbe esperimenti su snapshot o QA set diversi, e la leaderboard dovrebbe indovinare quali sono confrontabili.
2. Le stesse condizioni di esecuzione: le varianti girano nello stesso run, sulle stesse domande, nello stesso momento; conta per gli agenti esterni che cambiano nel tempo.
3. Il confronto domanda per domanda: test statistico appaiato, domande in cui le varianti divergono, vista affiancata.
4. Baseline e generazione delle varianti da una griglia di parametri.

I due modelli sono quasi equivalenti: "esperimento = una configurazione" più un'entità "confronto" è lo stesso modello con i nomi spostati. Quello attuale rende i vincoli strutturali. Per misurare una sola configurazione basta un esperimento con una variante.

Decisioni e punti aperti

Due decisioni sono prese e applicate; il resto è da chiudere prima o durante lo Sprint 07.

Decise:

- il retrieval gira nell'API; il worker possiede solo la scrittura degli indici;
- niente filtri sui documenti nel profilo di retrieval: restringere i documenti è compito dello snapshot.

Da decidere:

- [ ] metriche per chiamata e metriche di insieme sull'unione, più la metrica "chiamata del primo risultato rilevante"
- [ ] tabella per esecuzione × chunk con primo `call_index`, rank migliore, `in_context`, `cited`
- [ ] campo opzionale `context` nel contratto `/answer`, ed esclusione dichiarata se manca
- [ ] token di retrieval per esecuzione anticipato allo Sprint 07
- [ ] aggregazione delle ripetizioni: prima per domanda, poi fra domande
- [ ] filtri sui metadati per singola chiamata a `/retrieve`, o riformulare il requisito sui metadati dei documenti come filtri di retrieval
- [ ] operazione `fetch` accanto a `search`, con `kind` e `request` su `retrieval_calls`, stage `fetched`, componente `retrieval_system` per il puro retrieval; e se entra nello Sprint 07
- [ ] aggiornare il diagramma `evaluation-design.drawio`, che mostra ancora `metadata_filters`